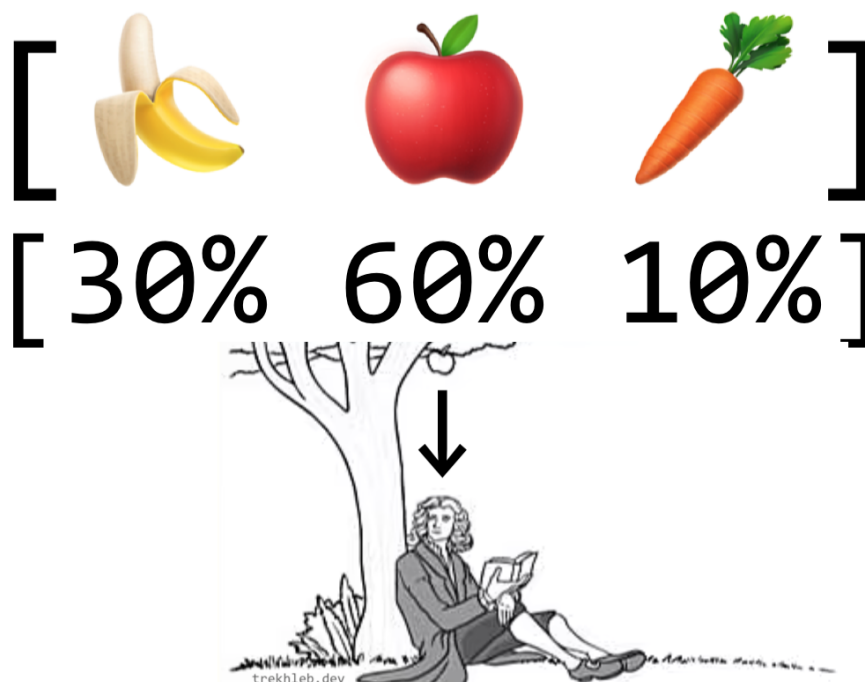


Weighted Random



What is "Weighted Random"

Let's say you have a list of **items**. Item could be anything. For example, we may have a list of fruits and vegetables that you like to eat: ['🍌', '🍏', '🥕'].

The list of **weights** represent the weight (or probability, or importance) of each item. Weights are numbers. For example, the weights like [3, 7, 1] would say that:

- you would like to eat 🍏 apples more often (7 out of $3 + 7 + 1 = 11$ times),
- then you would like to eat bananas 🍌 less often (only 3 out of 11 times),
- and the carrots 🥕 you really don't like (want to eat it only 1 out of 11 times).

If we speak in terms of probabilities than the weights list might be an array of floats that sum up to 1 (i.e. [0.1, 0.5, 0.2, 0.2]).

The **Weighted Random** in this case will be the function that will randomly return you the item from the list, and it will take each item's weight into account, so that items with the higher weight will be picked more often.

Example of the function interface:

```
const items = [ '🍌', '🍏', '🥕' ];
const weights = [ 3, 7, 1 ];

function weightedRandom(items, weights) {
  // implementation goes here ...
}
```

```
const nextSnackToEat = weightedRandom(items, weights); // Could be '🍏'
```

Applications of Weighted Random

- In [Genetic Algorithm](#) the weighted random is used during the "Selection" phase, when we need to select the fittest/strongest individuals based on their fitness score for mating and for producing the next stronger generation. You may find an **example** in the [Self-Parking Car in 500 Lines of Code](#) article.
- In [Recurrent Neural Networks \(RNN\)](#) when trying to decide what letter to choose next (to form the sentence) based on the next letter probability. You may find an **example** in the [Recipe Generation using Recurrent Neural Network \(RNN\)](#) Jupyter notebook.
- In [Nginx Load Balancing](#) to send HTTP requests more often to the servers with the higher weights.
- And more...

The Algorithm

The **straightforward approach** would be to:

1. Repeat each item in the list according to its weight.
2. Pick the random item from the list.

For example in our case with fruits and vegetables we could generate the following list of size $3 + 7 + 1 = 11$:

```
const items = [ '🍌', '🍏', '🥦' ];
const weights = [ 3, 7, 1 ];

// Repeating the items based on weights.
const weightedItems = [
  '🍌', '🍌', '🍌',
  '🍏', '🍏', '🍏', '🍏', '🍏', '🍏', '🍏',
  '🥦',
];

// And now just pick the random item from weightedItems array.
```

However, as you may see, this approach may require a lot of memory, in case if we have a lot of items to repeat in `weightedItems` list. Think of it as if you would need to repeat a string like `"some-random-string"` (18 bytes) a ten million times. You will need to allocate around 180Mb of additional memory space just for this array.

The **more efficient approach** would be to:

1. Prepare the list of cumulative weights for each item (i.e. the `cumulativeWeights` list which will have the same number of elements as the original `weights` list). In our case it will look like this:

```
cumulativeWeights = [3, 3 + 7, 3 + 7 + 1] = [3, 10, 11]
```

2. Generate the random number `randomNumber` from 0 to the highest cumulative weight value. In our case the random number will be in a range of `[0..11]`. Let's say that we have `randomNumber = 8`.
3. Go through the `cumulativeWeights` list from left to right and pick the first element which is higher or equal to the `randomNumber`. The index of such element we will use to pick the item from the `items` array.

The idea behind this approach is that the higher weights will "occupy" more numeric space. Therefore, there is a higher chance that the random number will fall into the "higher weight numeric bucket".

```
const weights =          [3, 7, 1];
const cumulativeWeights = [3, 10, 11];

// In a pseudo-representation we may think about the cumulativeWeights array
// like this.
const pseudoCumulativeWeights = [
  1, 2, 3,                // <-- [3] numbers
  4, 5, 6, 7, 8, 9, 10,    // <-- [7] numbers
  11,                     // <-- [1] number
];
```

Here is an example of how the `weightedRandom` function might be implemented:

```
/**
 * Picks the random item based on its weight.
 * The items with higher weight will be picked more often (with a higher
 * probability).
 *
 * For example:
 * - items = ['banana', 'orange', 'apple']
 * - weights = [0, 0.2, 0.8]
 * - weightedRandom(items, weights) in 80% of cases will return 'apple', in 20%
 * of cases will return
 * 'orange' and it will never return 'banana' (because probability of picking
 * the banana is 0%)
 *
 * @param {any[]} items
 * @param {number[]} weights
 * @returns {{item: any, index: number}}
 */
export default function weightedRandom(items, weights) {
  if (items.length !== weights.length) {
    throw new Error('Items and weights must be of the same size');
  }

  if (!items.length) {
    throw new Error('Items must not be empty');
  }

  // Preparing the cumulative weights array.
  // For example:
```

```
// - weights = [1, 4, 3]
// - cumulativeWeights = [1, 5, 8]
const cumulativeWeights = [];
for (let i = 0; i < weights.length; i += 1) {
  cumulativeWeights[i] = weights[i] + (cumulativeWeights[i - 1] || 0);
}

// Getting the random number in a range of [0...sum(weights)]
// For example:
// - weights = [1, 4, 3]
// - maxCumulativeWeight = 8
// - range for the random number is [0...8]
const maxCumulativeWeight = cumulativeWeights[cumulativeWeights.length - 1];
const randomNumber = maxCumulativeWeight * Math.random();

// Picking the random item based on its weight.
// The items with higher weight will be picked more often.
for (let itemIndex = 0; itemIndex < items.length; itemIndex += 1) {
  if (cumulativeWeights[itemIndex] >= randomNumber) {
    return {
      item: items[itemIndex],
      index: itemIndex,
    };
  }
}
}
```

Implementation

- Check the [weightedRandom.js](#) file for the implementation of the `weightedRandom()` function.
- Check the [weightedRandom.test.js](#) file for the tests-cases.